บทที่ 6

การควบคุมทิศทางการทำงานของโปรแกรม
การทำซ้ำ (Repetition)

Computer Engineering, Faculty of Engineering, Rajamangala University of Technology Thanyaburi

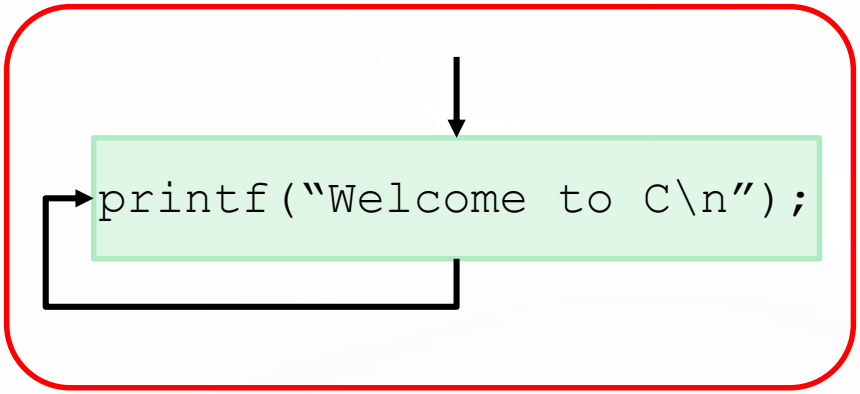
ทำไมต้องมีคำสั่งทำซ้ำ

หากเราต้องการเขียนโปรแกรม เพื่อแสดงข้อความเดิม 100 ครั้ง

Welcome to C

```
printf("Welcome to C\n");
printf("Welcome to C\n");
  ...
printf("Welcome to C\n");
printf("Welcome to C\n");
```

100 ครั้ง



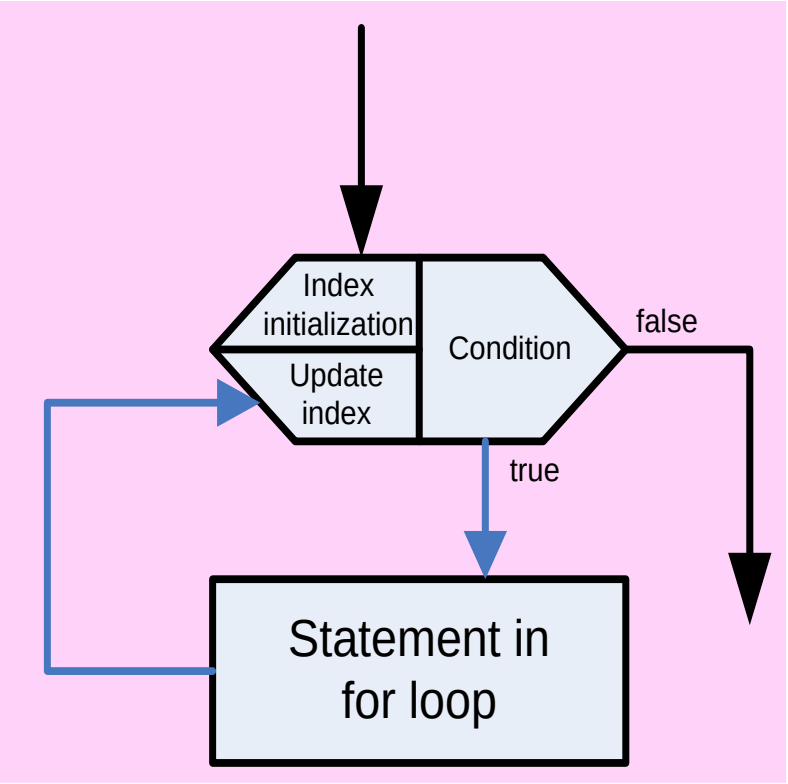
การทำงานซ้ำ

คำสั่งควบคุมการทำซ้ำในภาษา C

- คำสั่ง for
- คำสั่ง while
- คำสั่ง do-while

วงรอบ (Loop) หมายถึง ลำดับของคำสั่งในภาษาโปรแกรมที่ทำงานซ้ำต่อเนื่อง จนกว่าเงื่อนไขในการทำซ้ำนั้นจะสิ้นสุดลง

คำสั่ง for




```
for (index initialization;
Condition; Update Index)
      Statement ;
```

ประกอบด้วย 3 ส่วน คือ

1. **Index initialization** คือ การกำหนดค่าเริ่มต้นให้กับตัวแปรดัชนี (Index) ที่ใช้ควบคุมการทำงานในวงรอบ
2. **Condition** คือ เงื่อนไขในการเปรียบเทียบ หากเป็นจริง จะทำซ้ำ (Statement in for loop) หากเป็นเท็จ จะออกจากการทำซ้ำ
3. **Update index** คือ การเปลี่ยนค่าของตัวแปรดัชนี เพื่อให้ผลของการเปรียบเทียบเงื่อนไข (Condition) มีแนวโน้มที่จะเป็นเท็จ จะได้ออกจากการทำซ้ำ

ไวยากรณ์ของคำสั่ง for

```
for (index initialization; Condition; Update Index)
     Statement ;
```

```
for (index initialization; Condition; Update Index){
    Statement 1 ;
    Statement 2 ;
    Statement 3 ;
}
```

ตัวอย่าง คำสั่งแสดงคำว่า RMUTT ทั้งหมด 10 ครั้ง

1
```
for (n = 1; n <= 10; n++) printf("RMUTT\n");
```

2
```
for (n = 10; n >= 1; n--) printf("RMUTT\n");
```

คำสั่ง for

การใส่เครื่องหมาย ; หลังเงื่อนไขของคำสั่ง for จะไม่เกิดการทำซ้ำ

เช่น `for (n=0; n<=9; n++) ;`

สามารถประกาศตัวแปรในส่วนของ index initialization ได้หลายตัว

เช่น

```
for (int x=0,y=0; x+y<30; ++x, y+=5)
printf("%d+%d = %d\n", x, y, x+y);
```

ส่วนของ condition สามารถใส่นิพจน์การคำนวณได้

ตัวอย่างการใช้คำสั่ง for ซ้อนกัน (Nested for)

การแสดงสูตรคูณแม่ 2 ถึงสูตรคูณแม่ 12

```
int m, n;
for (m = 2; m <= 12; m++){
  for (n = 1; n <= 12; n++)
    printf("%d x %d = %d\n", m, n, m*n);
}
```

2 x 1 = 2

2 x 2 = 4

...

2 x 12 = 24

ตัวอย่างการใช้คำสั่ง for

การกำหนดค่าให้กับตัวแปรดัชนี (index) และเปลี่ยนค่าของตัวแปรดัชนีด้วยการรับค่าอักขระ โดยการใช้ฟังก์ชัน getche

```
1  #include <stdio.h>
2  #include <conio.h>
3  int main(void)
4  {
5    char c;
6    for (c=getche(); c!='q'; c=getche());
7    printf("\n q has been found \n");
8    return(0);
9  }
```

ผลการทำงาน

123abcq

q has been found

ตัวอย่างการใช้คำสั่ง for

```
1   #include <stdio.h>
2   int main(void){
3    int i;
4    for (i=1; i<100; i++){
5      if((i%2)== 0) continue;
6      printf("%d is odd number\n", i);
7      if (i>5) break;
8    }
9    return(0);
10  }
```

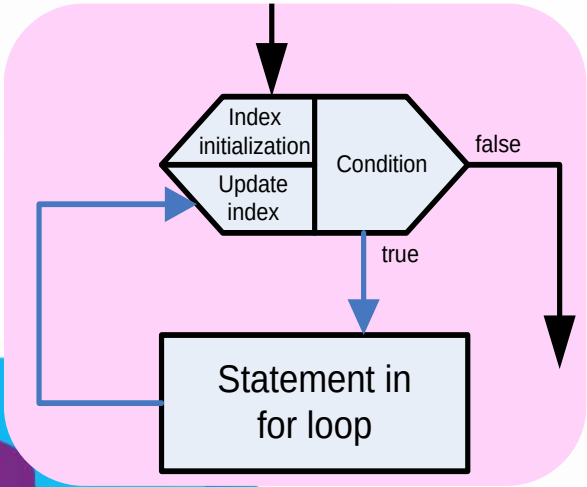
ผลการทำงาน

1 is odd number

3 is odd number

5 is odd number

7 is odd number



- คำสั่ง continue จะกระโดดไปยังจุด update ของคำสั่ง for
- คำสั่ง break จะกระโดดออกจาก loop

โดยไม่สนใจคำสั่งต่อไปในส่วนของการทำซ้ำ

ตัวอย่างการใช้คำสั่ง for



มีลวดหนามยาว 200 เมตร ต้องการล้อมรั้วให้มีพื้นที่มากที่สุด พื้นที่นั้นมีขนาดเท่าไหร่

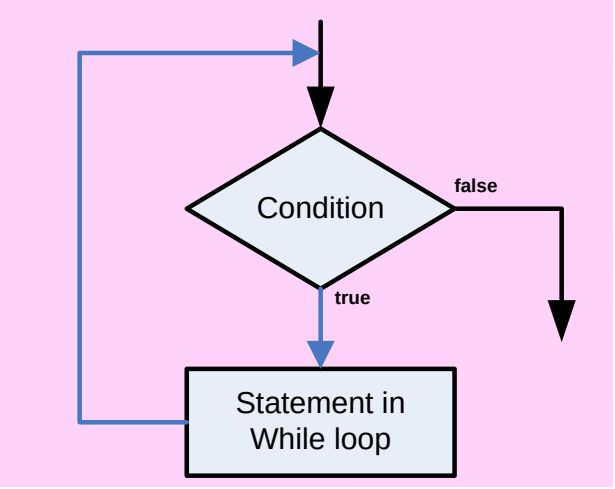
```
1   #include <stdio.h>
2   int main(void){
3     int x, area, max;
4     printf("Value of x      Area\n");
5     max = 0;
6     for (x=1; x<100; x++){
7       area = x*(100-x);
8       printf("%d          %d\n",x,area);
9       if (area > max) max = area;
10    }
11    printf("Maximum area is %d\n", max);
12    return(0);
13  }
```

ผลการทำงาน

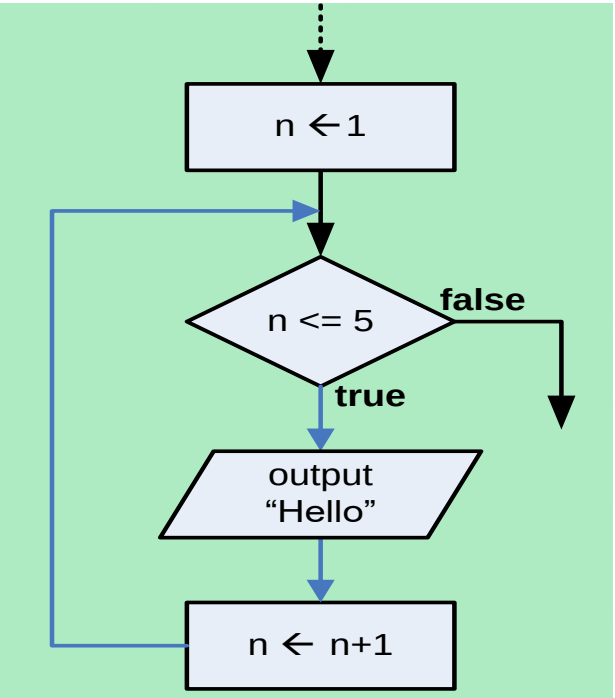
Value of x	Area
1	99
...	...
10	900
11	979
...	...
50	2500
...	...
58	2436
...	...
99	99

Maximum area is 2500

คำสั่ง while




```
while (condition)
   statement;
```



- **Condition** คือ เงื่อนไขในการทำซ้ำ ถ้าผลของการเปรียบเทียบเป็นจริง จึงจะทำคำสั่งในวงรอบ
- **Statement in While loop** คือ คำสั่งในวงรอบของคำสั่ง while
- ก่อนเข้าคำสั่ง while จะต้องกำหนดค่าเริ่มต้นให้กับตัวแปรดัชนี (index) ก่อน ในที่นี้ คือ คำสั่ง `n ← 1`
- ในวงรอบ จะต้องมีคำสั่งเพื่อแก้ไขค่าของตัวแปรดัชนี เพื่อให้ Condition มีแนวโน้มออกจากวงรอบได้ ในที่นี้ คือ คำสั่ง `n ← n+1`

Computer Engineering ● Faculty of Engineering ● Rajamangala University of Technology Thanyaburi

ตัวอย่างการใช้คำสั่ง while

```
n = 1;
while (n <= 5){
  printf("%d. RMUTT\n", n);
  n++;
}
```

คำสั่ง while สามารถทำซ้ำได้เพียงหนึ่งคำสั่ง
ถ้าต้องการให้ทำซ้ำมากกว่าหนึ่งคำสั่ง
ต้องใส่คำสั่งเหล่านั้นไว้ภายใต้เครื่องหมาย { }

ผลการทำงาน

1. RMUTT
2. RMUTT
3. RMUTT
4. RMUTT
5. RMUTT

ตัวอย่างการใช้คำสั่ง while

```
1   #include <stdio.h>
2   int main(void){
3     int num;
4     num = 1;
5     while (num <= 5){
6       printf("%d ", num);
7       num++;
8     }
9    return(0);
10  }
```

ผลการทำงาน

1 2 3 4 5

ตัวอย่างการใช้คำสั่ง while

```
1   #include <stdio.h>
2   int main(void){
3     int sum, n;
4     sum = 0;
5     n = 1;
6     while (n <= 10){
7       sum = sum+n;
8       n++;
9     }
10    printf("Summation of 1-10 is : %d", sum);
11    return(0);
12  }
```

ผลการทำงาน

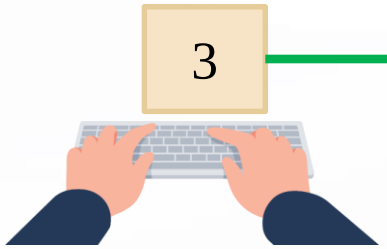
Summation of 1-10 is 55

ตัวอย่างการใช้คำสั่ง while

```
1   #include <stdio.h>
2   int main(void){
3     int sum, data;
4     sum = 0;
5     data = 1;
6     while (data != 0){
7       printf("Enter an integer (0 = stop) : ");
8       scanf("%d",&data);
9       sum = sum + data;
10    }
11    printf("The sum is : %d", sum);
12    return(0);
13  }
```

ผลการทำงาน

```
Enter an integer(0 = stop) : 3
Enter an integer(0 = stop) : 5
Enter an integer(0 = stop) : 6
Enter an integer(0 = stop) : 0
The sum is : 14
```



data	sum
1	0
3	3
5	8
6	14
0	14

ตัวอย่างการใช้คำสั่ง while

โปรแกรมหาผลบวกของเลขที่มีทศนิยม 1.0 + 0.9 + 0.8 + ... + 0.1

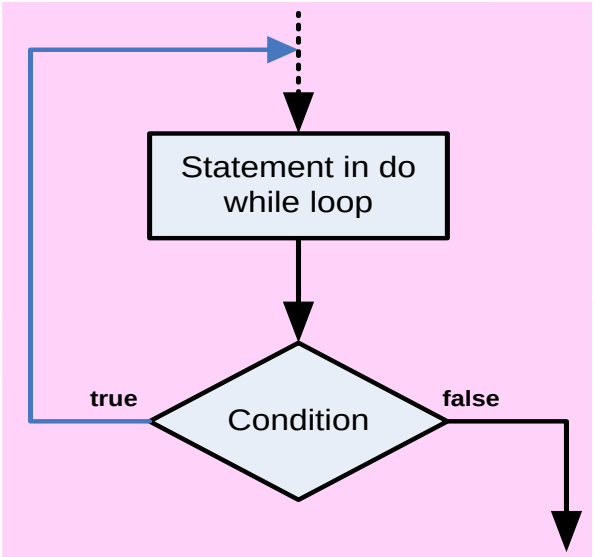
```
1   #include <stdio.h>
2   int main(void){
3     float sum, n;
4     sum = 0;
5     n = 1;
6     while (n > 0){
7       sum = sum+n;
8       n = n-0.1;
9     }
10    printf("Summation of 0.1 to 1 is: %f", sum);
11    return(0);
```

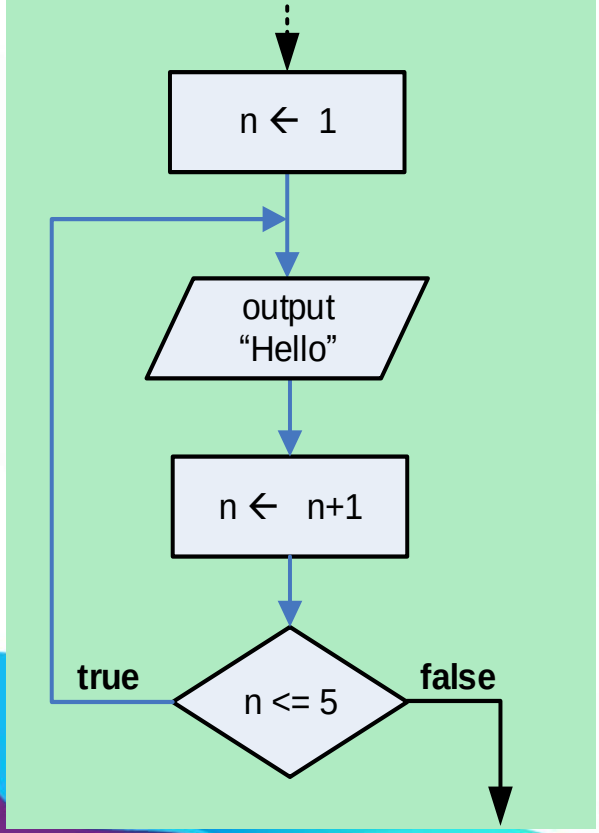
ผลการทำงาน

Summation of 1-0.1 is: 5.499999

ถ้าต้องการผลลัพธ์ 5.5 ต้องแก้เป็น %0.1f

⚠ ไม่แนะนำให้ใช้จำนวนจริง เปรียบเทียบกับเลขจำนวนเต็ม เช่น == 0, != 0 เป็นต้น





คำสั่ง do-while

```
do statement;
while (condition);
```

- **Condition** คือ เงื่อนไขในการทำซ้ำ ถ้าผลของการเปรียบเทียบเป็นจริง จะทำคำสั่งในวงรอบ ถ้าเป็นเท็จ จะออกนอกวงรอบ
- **Statement in do while loop** คือ คำสั่งในวงรอบของคำสั่ง do-while
- ก่อนเข้าคำสั่ง do-while จะต้องกำหนดค่าเริ่มต้นให้กับตัวแปรดัชนี (index) และในวงรอบ จะต้องมีคำสั่งเพื่อแก้ไขค่าของตัวแปรดัชนี เพื่อให้ Condition มีแนวโน้มออกจากวงรอบได้ เหมือนคำสั่ง while แต่คำสั่ง do-while จะทำคำสั่งในวงรอบอย่างน้อย 1 ครั้งเสมอ

ตัวอย่างการใช้คำสั่ง do-while

```
n = 1;
do{
  printf("%d. RMUTT\n", n);
  n++;
}while (n <= 5);
```

คำสั่ง do-while สามารถทำซ้ำได้เพียงหนึ่งคำสั่ง
ถ้าต้องการให้ทำซ้ำมากกว่าหนึ่งคำสั่ง
ต้องใส่คำสั่งเหล่านั้นไว้ภายใต้เครื่องหมาย { }

ผลการทำงาน

1. RMUTT
2. RMUTT
3. RMUTT
4. RMUTT
5. RMUTT

ตัวอย่างการใช้คำสั่ง do-while

โปรแกรมวนรับเลขจำนวนเต็ม 1 ตัวไปเรื่อย ๆ จนกว่าตัวเลขที่รับเข้าไปนั้นจะมีค่า 1 ถึง 4 จึงจะออกจากการวนรอบ

```
do {
    printf("****Menu****\n");
    printf(" 1. Show Odd  number\n");
    printf(" 2. Show Even number\n");
    printf(" 3. Count Up\n");
    printf(" 4. Count Down\n");
    printf(" Please enter number [1-4] : ");
    scanf("%d", &choice);
} while (choice < 0 || choice > 4);
```

สรุปการควบคุมการทำงานของโปรแกรมแบบทำซ้ำ

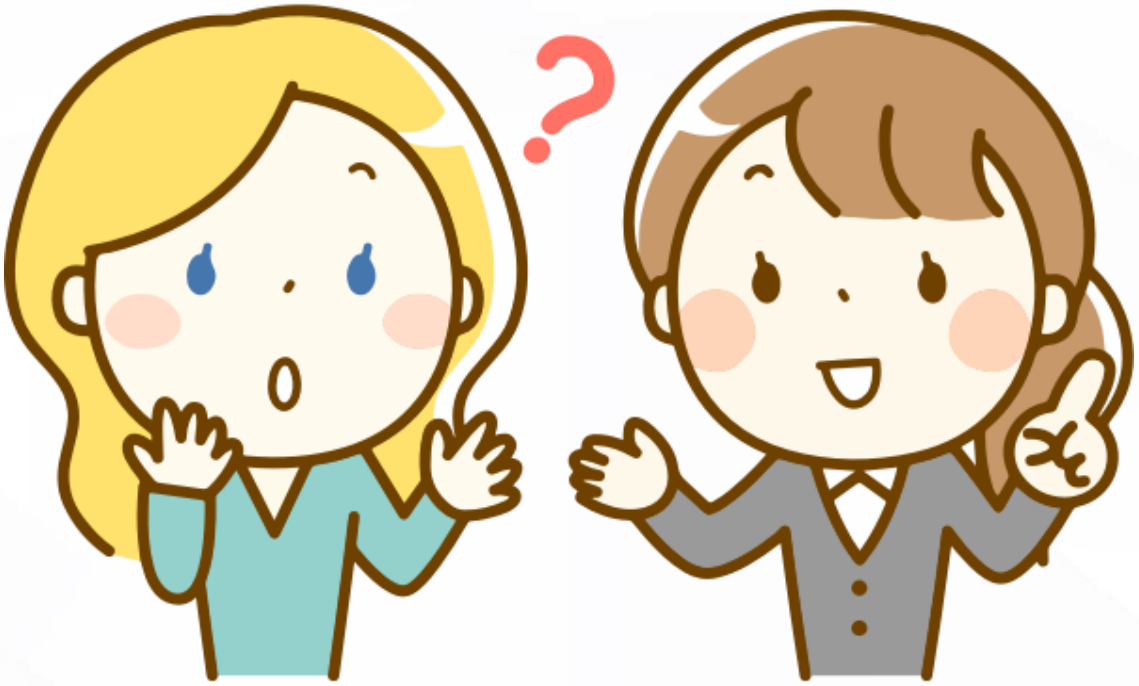
หลักในการทำซ้ำ ประกอบด้วยสิ่งสำคัญ 3 สิ่ง คือ

- การกำหนดค่าเริ่มต้นให้กับค่าดัชนี (Index)
- **เงื่อนไข (Condition)** ที่จะตัดสินว่า จะให้ทำซ้ำในวงรอบหรือไม่
- การปรับค่าดัชนี (Update index) ภายในวงรอบ

```
for(i=1; i<=5; i++){
   printf("RMUTT");
}
```

```
    i = 1;
    while (i <= 5){
      printf("RMUTT");
      i++;
    }
```

```
i = 1;
do{
   printf("RMUTT");
   i++;
}while (i<=5);
```



Questions & Answers